



UNIVERSIDAD DE LAS FUERZAS ARMADAS - ESPE

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

Ingeniería de la Seguridad del Software

Integrantes: Mateo Sosa

NRC: 30808

Fecha: June 12, 2026

Contents

Objetivos de Aprendizaje	4
Introducción	4
Marco Teórico	4
Topología de prueba	5
Desarrollo	6
Ejercicios de Cifrado	6
Ejercicios de Descifrado	10
Algoritmo en Python	15
Comunicación Cifrada entre Máquinas Virtuales	16
Ubuntu Desktop	16
Kali Linux	17
Repotenciar el Algoritmo en Python	18
Uso de CyberChef	21
Conclusiones	24
Referencias Bibliográficas	25

List of Figures

1	Cifrado Manual de Palabra “SOFTWARE”	6
2	Cifrado Manual de Palabra “SEGURIDAD”	7
3	Cifrado Manual de Palabra “CRIPTOGRAFIA”	8
4	Cifrado Manual de Palabra “I LOVE LINUX”	9
5	Cifrado Manual de Palabra “GOD IS LOVE”	10
6	Descifrado Manual de Palabra “KROD PXQGR”	11
7	Descifrado Manual de Palabra “FUBSWRVHJXULGDG”	12
8	Descifrado Manual de Palabra “SURJUDPDFLRQ HQ S BWKRQ” . . .	13
9	Descifrado Manual de Palabra “OLQXA LV DZHVRPH”	14

10	Descifrado Manual de Palabra “VDIH QHWZRUN”	15
11	Ejecución de Programa Python de Ejercicio 2	16
12	Escucha de Conexiones en Ubuntu Desktop	17
13	Envío del mensaje desde Kali Linux	17
14	Recibo del mensaje y conexión en Ubuntu Desktop	18
15	Ejecución del Programa Python de Ejercicio 4	21
16	Ejecución de Algoritmo “ROT13” en CyberChef	22
17	Ejecución de Algoritmo “Caesar Cipher” en CyberChef	22
18	Ejecución de Algoritmo “Base64 Encode” en CyberChef	23
19	Ejecución de Algoritmo “Base64 Decode” en CyberChef	24

List of Tables

Objetivos de Aprendizaje

Al finalizar esta práctica, usted será capaz de:

1. Comprender el proceso de encriptación y desencriptación mediante el cifrado César.
2. Entender el funcionamiento del algoritmo implementado en Python.
3. Repotenciar y mejorar el algoritmo básico del Código César.
4. Aprender a utilizar CyberChef para realizar operaciones criptográficas básicas.
5. Implementar comunicación cifrada entre máquinas virtuales utilizando Netcat (nc).

Introducción

La criptografía es una disciplina fundamental en la seguridad informática moderna. Desde los métodos clásicos hasta los sistemas criptográficos actuales, el objetivo principal consiste en proteger la información frente a accesos no autorizados.

Uno de los métodos históricos más conocidos es el Cifrado César, utilizado por Julio César para proteger mensajes militares. Aunque actualmente se considera inseguro frente a ataques modernos, sigue siendo un excelente mecanismo didáctico para comprender conceptos fundamentales de criptografía, cifrado y análisis criptográfico.

En esta práctica se implementará el algoritmo en Python, se analizará su funcionamiento y se utilizará la herramienta CyberChef para experimentar con procesos de cifrado y descifrado.

Marco Teórico

Criptología: es la ciencia que estudia los métodos de protección y análisis de información. Se divide principalmente en:

- Criptografía
- Criptoanálisis

Criptografía: consiste en desarrollar técnicas que permitan proteger la información mediante algoritmos de cifrado. Su objetivo es garantizar:

- Confidencialidad
- Integridad
- Autenticación
- No repudio

Criptoanálisis: El criptoanálisis estudia métodos para romper sistemas criptográficos o descubrir información sin conocer la clave utilizada.

Cifrado César: es un método de sustitución mono alfabética en el cual cada letra del mensaje original se desplaza un número fijo de posiciones en el alfabeto.

Fórmula del cifrado:

$$C(x) = (x + n) \bmod 26(1)$$

$$D(x) = (x - n) \bmod 26(2)$$

Donde:

- C: Cifrado
- D: Descifrado
- x: posición de la letra
- k: clave de desplazamiento
- 26: número de letras del alfabeto

CyberChef: es una herramienta web desarrollada por GCHQ utilizada para transformar, analizar, codificar, decodificar y cifrar información.

Características:

- Interfaz gráfica intuitiva
- Soporte para múltiples algoritmos
- Conversión de formatos
- Operaciones criptográficas
- Análisis de texto y datos.

Netcat (nc): es una herramienta de red utilizada para:

- Escaneo de puertos
- Transferencia de archivos
- Comunicación cliente-servidor
- Envío y recepción de mensajes TCP/UDP

En esta práctica se utilizará para enviar mensajes cifrados entre máquinas virtuales.

Topología de prueba

- Ubuntu Desktop: estación de trabajo del estudiante para desarrollo y pruebas.
- Metasploitable: Máquina objetivo de laboratorio.
- Kali Linux: máquina atacante / análisis y desarrollo

Desarrollo

Ejercicios de Cifrado

Cifrar manualmente las siguientes frases utilizando el Código César con un desplazamiento de 3 posiciones:

- SOFTWARE

```
Texto original: SOFTWARE
Desplazamiento: 3

Proceso de cifrado:
-----

Carácter 1: 'S'
Posición original: 18
 $(18 + 3) \% 26 = 21$ 
Carácter cifrado: 'V'

Carácter 2: 'O'
Posición original: 14
 $(14 + 3) \% 26 = 17$ 
Carácter cifrado: 'R'

Carácter 3: 'F'
Posición original: 5
 $(5 + 3) \% 26 = 8$ 
Carácter cifrado: 'I'

Carácter 4: 'T'
Posición original: 19
 $(19 + 3) \% 26 = 22$ 
Carácter cifrado: 'W'

Carácter 5: 'W'
Posición original: 22
 $(22 + 3) \% 26 = 25$ 
Carácter cifrado: 'Z'

Carácter 6: 'A'
Posición original: 0
 $(0 + 3) \% 26 = 3$ 
Carácter cifrado: 'D'

Carácter 7: 'R'
Posición original: 17
 $(17 + 3) \% 26 = 20$ 
Carácter cifrado: 'U'

Carácter 8: 'E'
Posición original: 4
 $(4 + 3) \% 26 = 7$ 
Carácter cifrado: 'H'

-----
Resultado final: VRIWZDUH
```

Figure 1: Cifrado Manual de Palabra “SOFTWARE”

- SEGURIDAD

```
Texto original: SEGURIDAD
Desplazamiento: 3
```

```
Proceso de cifrado:
```

```
-----
```

```
Carácter 1: 'S'
Posición original: 18
 $(18 + 3) \% 26 = 21$ 
Carácter cifrado: 'V'
```

```
Carácter 2: 'E'
Posición original: 4
 $(4 + 3) \% 26 = 7$ 
Carácter cifrado: 'H'
```

```
Carácter 3: 'G'
Posición original: 6
 $(6 + 3) \% 26 = 9$ 
Carácter cifrado: 'J'
```

```
Carácter 4: 'U'
Posición original: 20
 $(20 + 3) \% 26 = 23$ 
Carácter cifrado: 'X'
```

```
Carácter 5: 'R'
Posición original: 17
 $(17 + 3) \% 26 = 20$ 
Carácter cifrado: 'U'
```

```
Carácter 6: 'I'
Posición original: 8
 $(8 + 3) \% 26 = 11$ 
Carácter cifrado: 'L'
```

```
Carácter 7: 'D'
Posición original: 3
 $(3 + 3) \% 26 = 6$ 
Carácter cifrado: 'G'
```

```
Carácter 8: 'A'
Posición original: 0
 $(0 + 3) \% 26 = 3$ 
Carácter cifrado: 'D'
```

```
Carácter 9: 'D'
Posición original: 3
 $(3 + 3) \% 26 = 6$ 
Carácter cifrado: 'G'
```

```
-----
```

```
Resultado final: VHJXULGDG
```

Figure 2: Cifrado Manual de Palabra “SEGURIDAD”

- CRIPTOGRAFIA

```
Texto original: CRIPTOGRAFIA
Desplazamiento: 3
```

```
Proceso de cifrado:
```

```
-----
```

```
Carácter 1: 'C'
Posición original: 2
 $(2 + 3) \% 26 = 5$ 
Carácter cifrado: 'F'
```

```
Carácter 2: 'R'
Posición original: 17
 $(17 + 3) \% 26 = 20$ 
Carácter cifrado: 'U'
```

```
Carácter 3: 'I'
Posición original: 8
 $(8 + 3) \% 26 = 11$ 
Carácter cifrado: 'L'
```

```
Carácter 4: 'P'
Posición original: 15
 $(15 + 3) \% 26 = 18$ 
Carácter cifrado: 'S'
```

```
Carácter 5: 'T'
Posición original: 19
 $(19 + 3) \% 26 = 22$ 
Carácter cifrado: 'W'
```

```
Carácter 6: 'O'
Posición original: 14
 $(14 + 3) \% 26 = 17$ 
Carácter cifrado: 'R'
```

```
Carácter 7: 'G'
Posición original: 6
 $(6 + 3) \% 26 = 9$ 
Carácter cifrado: 'J'
```

```
Carácter 8: 'R'
Posición original: 17
 $(17 + 3) \% 26 = 20$ 
Carácter cifrado: 'U'
```

```
Carácter 9: 'A'
Posición original: 0
 $(0 + 3) \% 26 = 3$ 
Carácter cifrado: 'D'
```

```
Carácter 10: 'F'
Posición original: 5
 $(5 + 3) \% 26 = 8$ 
Carácter cifrado: 'I'
```

```
Carácter 11: 'I'
Posición original: 8
 $(8 + 3) \% 26 = 11$ 
Carácter cifrado: 'L'
```

Figure 3: Cifrado Manual de Palabra “CRIPTOGRAFIA”

- I LOVE LINUX


```

Carácter 1: 'I'
Posición original: 8
 $(8 + 3) \% 26 = 11$ 
Carácter cifrado: 'L'

Carácter 2: ' '
No es una letra, se conserva igual.

Carácter 3: 'L'
Posición original: 11
 $(11 + 3) \% 26 = 14$ 
Carácter cifrado: 'O'

Carácter 4: 'O'
Posición original: 14
 $(14 + 3) \% 26 = 17$ 
Carácter cifrado: 'R'

Carácter 5: 'V'
Posición original: 21
 $(21 + 3) \% 26 = 24$ 
Carácter cifrado: 'Y'

Carácter 6: 'E'
Posición original: 4
 $(4 + 3) \% 26 = 7$ 
Carácter cifrado: 'H'

Carácter 7: ' '
No es una letra, se conserva igual.

Carácter 8: 'L'
Posición original: 11
 $(11 + 3) \% 26 = 14$ 
Carácter cifrado: 'O'

Carácter 9: 'I'
Posición original: 8
 $(8 + 3) \% 26 = 11$ 
Carácter cifrado: 'L'

Carácter 10: 'N'
Posición original: 13
 $(13 + 3) \% 26 = 16$ 
Carácter cifrado: 'Q'

Carácter 11: 'U'
Posición original: 20
 $(20 + 3) \% 26 = 23$ 
Carácter cifrado: 'X'

Carácter 12: 'X'
Posición original: 23
 $(23 + 3) \% 26 = 0$ 
Carácter cifrado: 'A'

-----
Resultado final: L ORYH OLQXA

```

Figure 4: Cifrado Manual de Palabra “I LOVE LINUX”

- GOD IS LOVE

```

Texto original: GOD IS LOVE
Desplazamiento: 3

Proceso de cifrado:
-----

Carácter 1: 'G'
Posición original: 6
 $(6 + 3) \% 26 = 9$ 
Carácter cifrado: 'J'

Carácter 2: 'O'
Posición original: 14
 $(14 + 3) \% 26 = 17$ 
Carácter cifrado: 'R'

Carácter 3: 'D'
Posición original: 3
 $(3 + 3) \% 26 = 6$ 
Carácter cifrado: 'G'

Carácter 4: ' '
No es una letra, se conserva igual.

Carácter 5: 'I'
Posición original: 8
 $(8 + 3) \% 26 = 11$ 
Carácter cifrado: 'L'

Carácter 6: 'S'
Posición original: 18
 $(18 + 3) \% 26 = 21$ 
Carácter cifrado: 'V'

Carácter 7: ' '
No es una letra, se conserva igual.

Carácter 8: 'L'
Posición original: 11
 $(11 + 3) \% 26 = 14$ 
Carácter cifrado: 'O'

Carácter 9: 'O'
Posición original: 14
 $(14 + 3) \% 26 = 17$ 
Carácter cifrado: 'R'

Carácter 10: 'V'
Posición original: 21
 $(21 + 3) \% 26 = 24$ 
Carácter cifrado: 'Y'

Carácter 11: 'E'
Posición original: 4
 $(4 + 3) \% 26 = 7$ 
Carácter cifrado: 'H'

-----
Resultado final: JRG LV ORYH

```

Figure 5: Cifrado Manual de Palabra “GOD IS LOVE”

Ejercicios de Descifrado

Descifrar los siguientes mensajes aplicando manualmente el desplazamiento inverso del Código César:

- KROD PXQGR

```

Texto cifrado: KROD PXQGR
Desplazamiento: 3

Proceso de descifrado:
-----

Carácter 1: 'K'
Posición cifrada: 10
 $(10 - 3) \% 26 = 7$ 
Carácter descifrado: 'H'

Carácter 2: 'R'
Posición cifrada: 17
 $(17 - 3) \% 26 = 14$ 
Carácter descifrado: 'O'

Carácter 3: 'O'
Posición cifrada: 14
 $(14 - 3) \% 26 = 11$ 
Carácter descifrado: 'L'

Carácter 4: 'D'
Posición cifrada: 3
 $(3 - 3) \% 26 = 0$ 
Carácter descifrado: 'A'

Carácter 5: ' '
No es una letra, se conserva igual.

Carácter 6: 'P'
Posición cifrada: 15
 $(15 - 3) \% 26 = 12$ 
Carácter descifrado: 'M'

Carácter 7: 'X'
Posición cifrada: 23
 $(23 - 3) \% 26 = 20$ 
Carácter descifrado: 'U'

Carácter 8: 'Q'
Posición cifrada: 16
 $(16 - 3) \% 26 = 13$ 
Carácter descifrado: 'N'

Carácter 9: 'G'
Posición cifrada: 6
 $(6 - 3) \% 26 = 3$ 
Carácter descifrado: 'D'

Carácter 10: 'R'
Posición cifrada: 17
 $(17 - 3) \% 26 = 14$ 
Carácter descifrado: 'O'

-----
Mensaje original: HOLA MUNDO

```

Figure 6: Descifrado Manual de Palabra “KROD PXQGR”

- FUBSWRVHJXULGDG

```

Carácter 5: 'W'
Posición cifrada: 22
 $(22 - 3) \% 26 = 19$ 
Carácter descifrado: 'T'

Carácter 6: 'R'
Posición cifrada: 17
 $(17 - 3) \% 26 = 14$ 
Carácter descifrado: 'O'

Carácter 7: 'V'
Posición cifrada: 21
 $(21 - 3) \% 26 = 18$ 
Carácter descifrado: 'S'

Carácter 8: 'H'
Posición cifrada: 7
 $(7 - 3) \% 26 = 4$ 
Carácter descifrado: 'E'

Carácter 9: 'J'
Posición cifrada: 9
 $(9 - 3) \% 26 = 6$ 
Carácter descifrado: 'G'

Carácter 10: 'X'
Posición cifrada: 23
 $(23 - 3) \% 26 = 20$ 
Carácter descifrado: 'U'

Carácter 11: 'U'
Posición cifrada: 20
 $(20 - 3) \% 26 = 17$ 
Carácter descifrado: 'R'

Carácter 12: 'L'
Posición cifrada: 11
 $(11 - 3) \% 26 = 8$ 
Carácter descifrado: 'I'

Carácter 13: 'G'
Posición cifrada: 6
 $(6 - 3) \% 26 = 3$ 
Carácter descifrado: 'D'

Carácter 14: 'D'
Posición cifrada: 3
 $(3 - 3) \% 26 = 0$ 
Carácter descifrado: 'A'

Carácter 15: 'G'
Posición cifrada: 6
 $(6 - 3) \% 26 = 3$ 
Carácter descifrado: 'D'

-----
Mensaje original: CRYPTOSEGURIDAD

```

Figure 7: Descifrado Manual de Palabra “FUBSWRVHJXULGDG”

- SURJUDPDFLRQ HQ S BWKRQ

```

Carácter 12: 'Q'
Posición cifrada: 16
 $(16 - 3) \% 26 = 13$ 
Carácter descifrado: 'N'

Carácter 13: ' '
No es una letra, se conserva igual.

Carácter 14: 'H'
Posición cifrada: 7
 $(7 - 3) \% 26 = 4$ 
Carácter descifrado: 'E'

Carácter 15: 'Q'
Posición cifrada: 16
 $(16 - 3) \% 26 = 13$ 
Carácter descifrado: 'N'

Carácter 16: ' '
No es una letra, se conserva igual.

Carácter 17: 'S'
Posición cifrada: 18
 $(18 - 3) \% 26 = 15$ 
Carácter descifrado: 'P'

Carácter 18: ' '
No es una letra, se conserva igual.

Carácter 19: 'B'
Posición cifrada: 1
 $(1 - 3) \% 26 = 24$ 
Carácter descifrado: 'Y'

Carácter 20: 'W'
Posición cifrada: 22
 $(22 - 3) \% 26 = 19$ 
Carácter descifrado: 'T'

Carácter 21: 'K'
Posición cifrada: 10
 $(10 - 3) \% 26 = 7$ 
Carácter descifrado: 'H'

Carácter 22: 'R'
Posición cifrada: 17
 $(17 - 3) \% 26 = 14$ 
Carácter descifrado: 'O'

Carácter 23: 'Q'
Posición cifrada: 16
 $(16 - 3) \% 26 = 13$ 
Carácter descifrado: 'N'

-----
Mensaje original: PROGRAMACION EN PYTHON

```

Figure 8: Descifrado Manual de Palabra “SURJUDPDFLRQ HQ S BWKRQ”

- OLQXA LV DZHVRPH

```

(0 - 3) % 26 = 23
Carácter descifrado: 'X'

Carácter 6: ' '
No es una letra, se conserva igual.

Carácter 7: 'L'
Posición cifrada: 11
(11 - 3) % 26 = 8
Carácter descifrado: 'I'

Carácter 8: 'V'
Posición cifrada: 21
(21 - 3) % 26 = 18
Carácter descifrado: 'S'

Carácter 9: ' '
No es una letra, se conserva igual.

Carácter 10: 'D'
Posición cifrada: 3
(3 - 3) % 26 = 0
Carácter descifrado: 'A'

Carácter 11: 'Z'
Posición cifrada: 25
(25 - 3) % 26 = 22
Carácter descifrado: 'W'

Carácter 12: 'H'
Posición cifrada: 7
(7 - 3) % 26 = 4
Carácter descifrado: 'E'

Carácter 13: 'V'
Posición cifrada: 21
(21 - 3) % 26 = 18
Carácter descifrado: 'S'

Carácter 14: 'R'
Posición cifrada: 17
(17 - 3) % 26 = 14
Carácter descifrado: 'O'

Carácter 15: 'P'
Posición cifrada: 15
(15 - 3) % 26 = 12
Carácter descifrado: 'M'

Carácter 16: 'H'
Posición cifrada: 7
(7 - 3) % 26 = 4
Carácter descifrado: 'E'

-----
Mensaje original: LINUX IS AWESOME

```

Figure 9: Descifrado Manual de Palabra “OLQXA LV DZHVRPH”

- VDIH QHWZRUN

```

Carácter 2: 'D'
Posición cifrada: 3
 $(3 - 3) \% 26 = 0$ 
Carácter descifrado: 'A'

Carácter 3: 'I'
Posición cifrada: 8
 $(8 - 3) \% 26 = 5$ 
Carácter descifrado: 'F'

Carácter 4: 'H'
Posición cifrada: 7
 $(7 - 3) \% 26 = 4$ 
Carácter descifrado: 'E'

Carácter 5: ' '
No es una letra, se conserva igual.

Carácter 6: 'Q'
Posición cifrada: 16
 $(16 - 3) \% 26 = 13$ 
Carácter descifrado: 'N'

Carácter 7: 'H'
Posición cifrada: 7
 $(7 - 3) \% 26 = 4$ 
Carácter descifrado: 'E'

Carácter 8: 'W'
Posición cifrada: 22
 $(22 - 3) \% 26 = 19$ 
Carácter descifrado: 'T'

Carácter 9: 'Z'
Posición cifrada: 25
 $(25 - 3) \% 26 = 22$ 
Carácter descifrado: 'W'

Carácter 10: 'R'
Posición cifrada: 17
 $(17 - 3) \% 26 = 14$ 
Carácter descifrado: 'O'

Carácter 11: 'U'
Posición cifrada: 20
 $(20 - 3) \% 26 = 17$ 
Carácter descifrado: 'R'

Carácter 12: 'N'
Posición cifrada: 13
 $(13 - 3) \% 26 = 10$ 
Carácter descifrado: 'K'

-----
Mensaje original: SAFE NETWORK

```

Figure 10: Descifrado Manual de Palabra “VDIH QHWZRUN”

Algoritmo en Python

Código Base:

```

def cifrado_cesar(texto, desplazamiento):
    resultado = ""

    for caracter in texto:

```

```

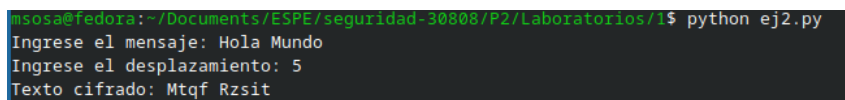
    if character.isalpha():
        base = ord('A') if character.isupper() else ord('a')
        nuevo_caracter = chr(
            (ord(character) - base + desplazamiento) % 26 + base
        )

        resultado += nuevo_caracter
    else:
        resultado += character
return resultado

mensaje = input("Ingrese el mensaje: ")
clave = int(input("Ingrese el desplazamiento: "))

texto_cifrado = cifrado_cesar(mensaje, clave)
print("Texto cifrado:", texto_cifrado)

```



```

msosa@fedora:~/Documents/ESPE/seguridad-30808/P2/Laboratorios/1$ python ej2.py
Ingrese el mensaje: Hola Mundo
Ingrese el desplazamiento: 5
Texto cifrado: Mtqf Rzsit

```

Figure 11: Ejecución de Programa Python de Ejercicio 2

Analizar el código y responder:

1. ¿Qué función cumple ord()?

Convierte un carácter en su código numérico Unicode (para letras inglesas coincide con ASCII).

2. ¿Qué función cumple chr()?

Realiza la operación inversa de ord(): convierte un número Unicode en un carácter.

3. ¿Qué representa % 26?

El operador % es el módulo (resto de una división).

4. ¿Cómo funciona el desplazamiento?

El desplazamiento indica cuántas posiciones debe moverse cada letra dentro del alfabeto.

5. ¿Qué ocurre con símbolos y espacios?

Los símbolos, espacios, números y signos de puntuación no se cifran.

Comunicación Cifrada entre Máquinas Virtuales

Ubuntu Desktop

Escuchar conexiones:

```
nc -lvnp 4444
```

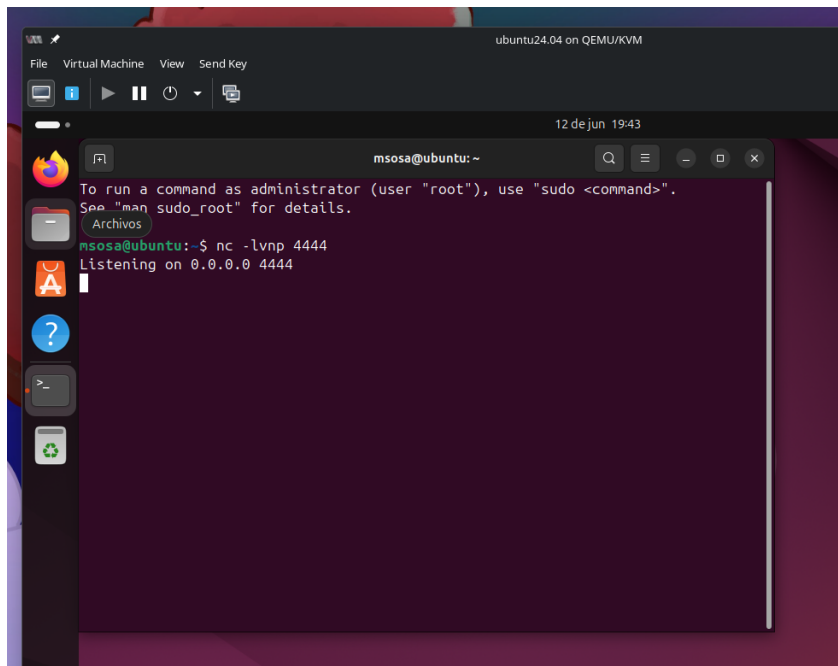



Figure 12: Escucha de Conexiones en Ubuntu Desktop

Kali Linux

Enviar mensaje:

echo "PHQVDMH HQFULSWDGR" | nc <IP_UBUNTU> 4444

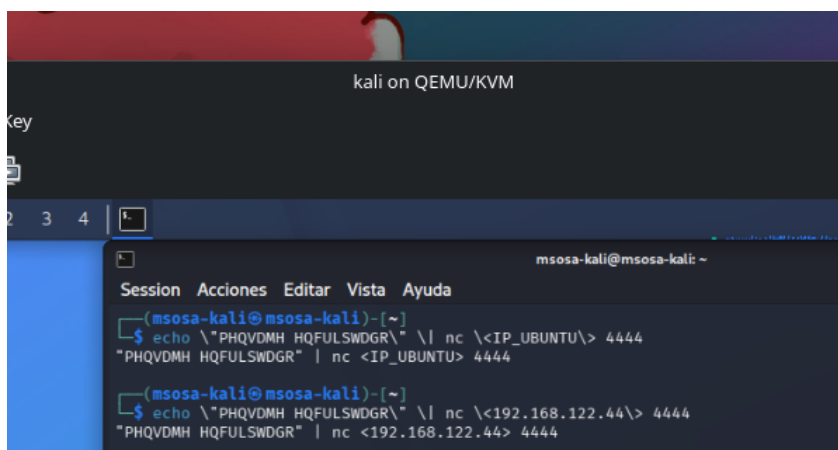


Figure 13: Envío del mensaje desde Kali Linux

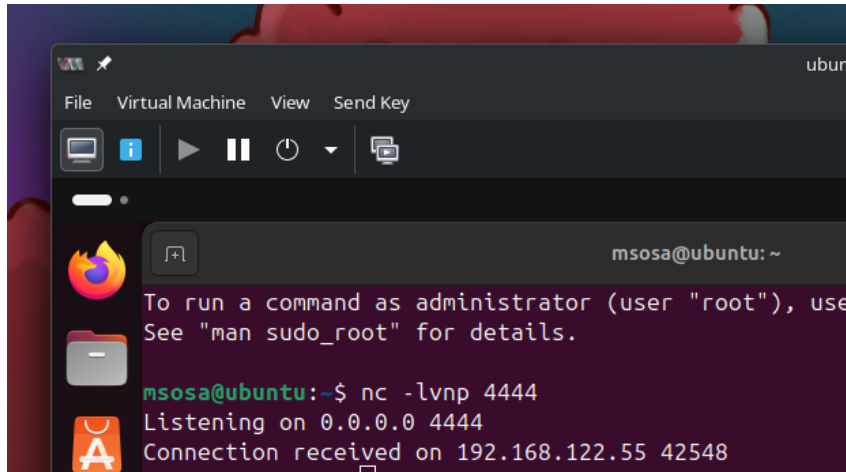


Figure 14: Recibo del mensaje y conexión en Ubuntu Desktop

Repotenciar el Algoritmo en Python

Modificar el programa para:

- Permitir cifrado y descifrado.
- Incorporar menú interactivo.
- Validar entradas incorrectas.
- Soportar mayúsculas y minúsculas.
- Manejar caracteres especiales.
- Ejecutar las pruebas de evaluación y validación correspondientes.

El nuevo programa es el siguiente:

```
def cifrado_cesar(texto, desplazamiento):
    resultado = ""

    for caracter in texto:
        if caracter.isalpha():
            base = ord('A') if caracter.isupper() else ord('a')

            nuevo_caracter = chr(
                (ord(caracter) - base + desplazamiento) % 26 + base
            )

            resultado += nuevo_caracter
        else:
            # Mantener espacios, números y símbolos
            resultado += caracter

    return resultado
```

```

def descifrado_cesar(texto, desplazamiento):
    return cifrado_cesar(texto, -desplazamiento)

def obtener_desplazamiento():
    while True:
        try:
            valor = int(input("Ingrese el desplazamiento (entero): "))
            return valor
        except ValueError:
            print("Error: debe ingresar un número entero.")

def pruebas():
    print("\n=== PRUEBAS AUTOMÁTICAS ===")

    mensaje = "Hola Mundo"
    clave = 3

    cifrado = cifrado_cesar(mensaje, clave)
    descifrado = descifrado_cesar(cifrado, clave)

    print(f"Mensaje original : {mensaje}")
    print(f"Cifrado           : {cifrado}")
    print(f"Descifrado         : {descifrado}")

    assert descifrado == mensaje

    # Prueba de mayúsculas y minúsculas
    assert cifrado_cesar("ABC xyz", 2) == "CDE zab"

    # Prueba de caracteres especiales
    assert cifrado_cesar("Hola, Mundo! 123", 5) == "Mtqf, Rzsit! 123"

    print("Todas las pruebas fueron superadas.\n")

def menu():
    while True:
        print("\n===== CIFRADO CÉSAR =====")
        print("1. Cifrar mensaje")
        print("2. Descifrar mensaje")
        print("3. Ejecutar pruebas")
        print("4. Salir")

        opcion = input("Seleccione una opción: ")

        if opcion == "1":

```

```

    mensaje = input("Ingrese el mensaje: ")
    clave = obtener_desplazamiento()

    resultado = cifrado_cesar(mensaje, clave)
    print("Mensaje cifrado:", resultado)

elif opcion == "2":
    mensaje = input("Ingrese el mensaje cifrado: ")
    clave = obtener_desplazamiento()

    resultado = descifrado_cesar(mensaje, clave)
    print("Mensaje descifrado:", resultado)

elif opcion == "3":
    pruebas()

elif opcion == "4":
    print("Programa finalizado.")
    break

else:
    print("Opción inválida. Intente nuevamente.")

if __name__ == "__main__":
    menu()

```

```

===== CIFRADO CÉSAR =====
1. Cifrar mensaje
2. Descifrar mensaje
3. Ejecutar pruebas
4. Salir
Seleccione una opción: 1
Ingrese el mensaje: HolaMundo
Ingrese el desplazamiento (entero): 5
Mensaje cifrado: MtqfRzsit

===== CIFRADO CÉSAR =====
1. Cifrar mensaje
2. Descifrar mensaje
3. Ejecutar pruebas
4. Salir
Seleccione una opción: 2
Ingrese el mensaje cifrado: MtqfRzsit
Ingrese el desplazamiento (entero): 5
Mensaje descifrado: HolaMundo

===== CIFRADO CÉSAR =====
1. Cifrar mensaje
2. Descifrar mensaje
3. Ejecutar pruebas
4. Salir
Seleccione una opción: 3

=== PRUEBAS AUTOMÁTICAS ===
Mensaje original : Hola Mundo
Cifrado          : Krod Pxqgr
Descifrado       : Hola Mundo
Todas las pruebas fueron superadas.

===== CIFRADO CÉSAR =====
1. Cifrar mensaje
2. Descifrar mensaje
3. Ejecutar pruebas
4. Salir
Seleccione una opción: 4
Programa finalizado.

```

Figure 15: Ejecución del Programa Python de Ejercicio 4

Uso de CyberChef

Ingresa a: <https://gchq.github.io/CyberChef/>

Realizar las siguientes operaciones:

1. ROT13

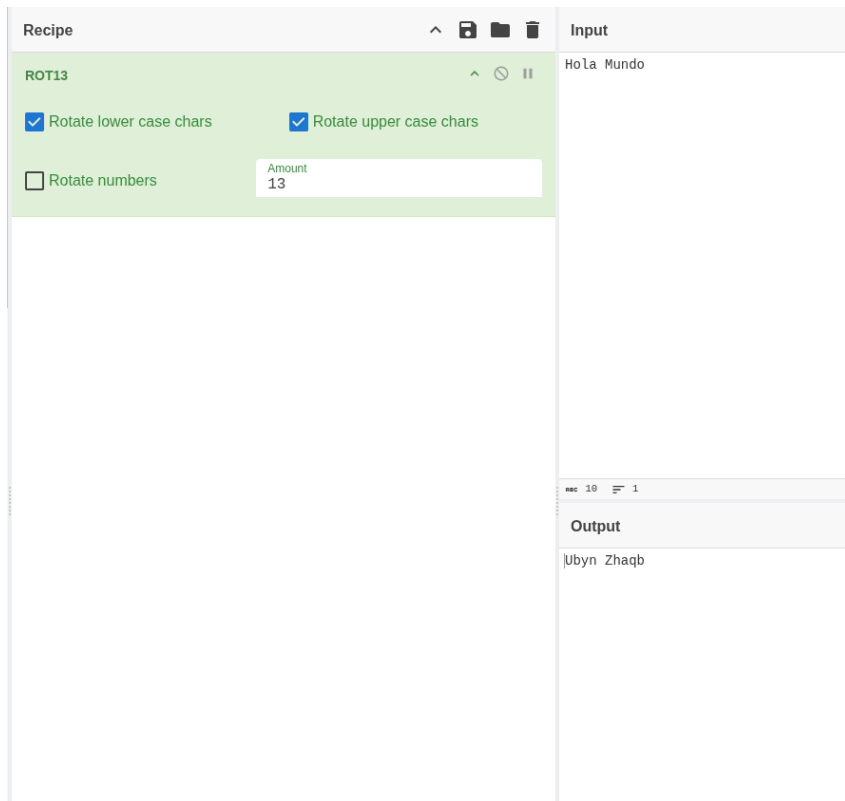


Figure 16: Ejecución de Algoritmo “ROT13” en CyberChef

2. Caesar Cipher

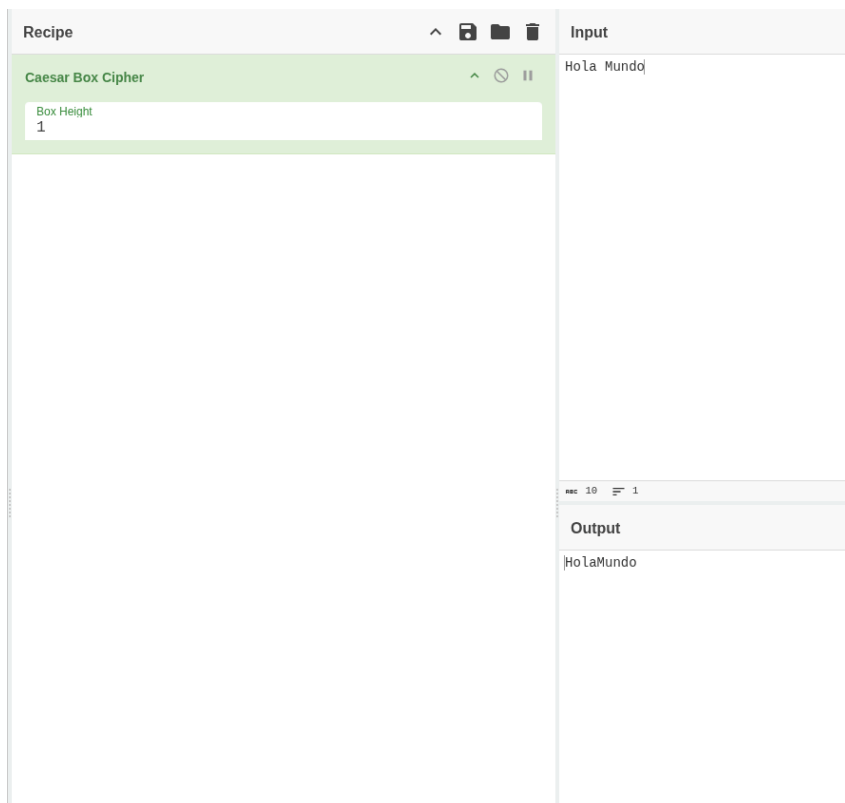


Figure 17: Ejecución de Algoritmo “Caesar Cipher” en CyberChef

3. Base64 Encode

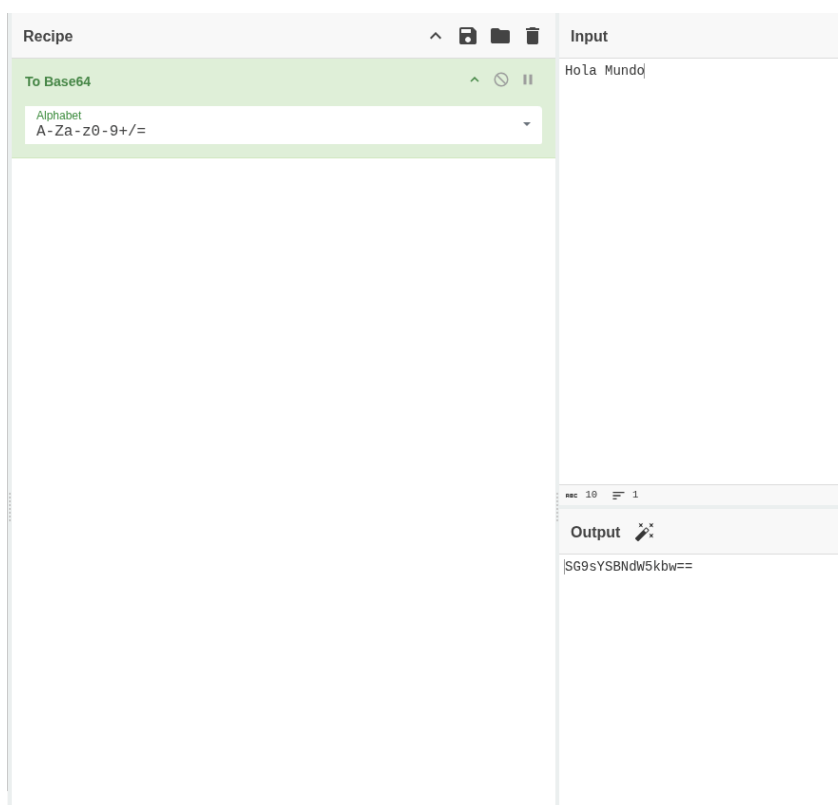


Figure 18: Ejecución de Algoritmo “Base64 Encode” en CyberChef

4. Base64 Decode

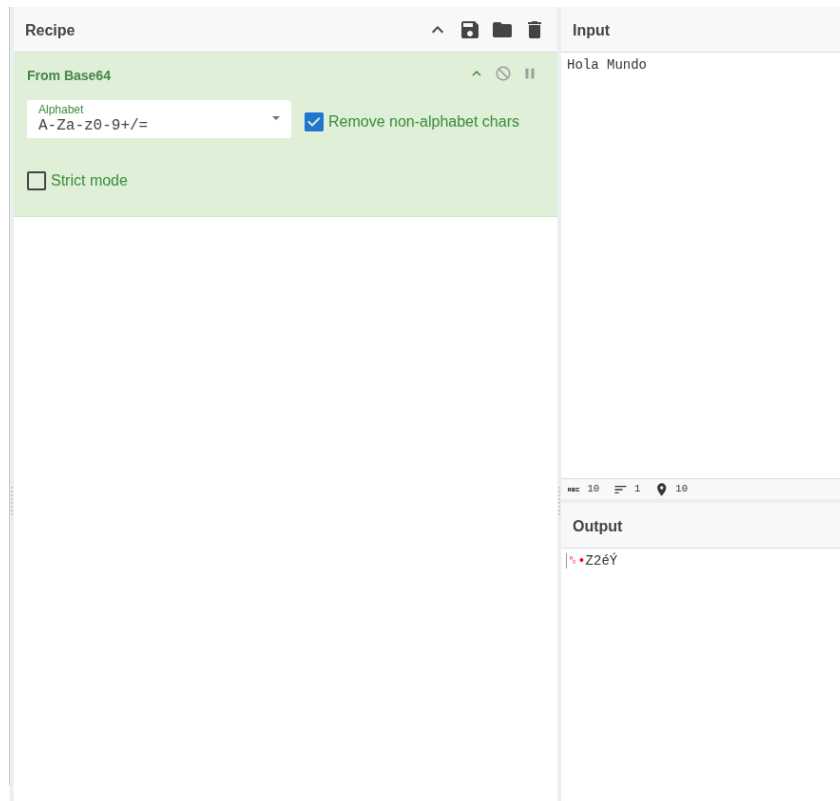


Figure 19: Ejecución de Algoritmo “Base64 Decode” en CyberChef

Investigar:

1. ¿Qué otras herramientas ofrecen CyberChef?

CyberChef ofrece herramientas de cifrado, descifrado, codificación, decodificación, cálculo de hashes y conversión de formatos.

2. ¿Qué ventajas presenta?

CyberChef es fácil de usar, gratuito y permite realizar múltiples operaciones sin programar.

3. ¿Qué limitaciones tiene?

CyberChef puede ser lento con archivos grandes y no reemplaza herramientas especializadas.

4. ¿Puede utilizarse en análisis forense?

Sí, CyberChef puede utilizarse para analizar, decodificar y transformar evidencias digitales.

Conclusiones

- El cifrado César permite comprender fundamentos esenciales de la criptografía clásica.
- Python facilita la implementación y experimentación de algoritmos criptográficos.
- Las máquinas virtuales permiten realizar pruebas seguras en entornos controlados.

- Netcat facilita comprender procesos básicos de comunicación en red.
- CyberChef constituye una herramienta útil para análisis y transformación criptográfica.
- Los algoritmos clásicos presentan vulnerabilidades importantes frente a ataques modernos.

Referencias Bibliográficas

- Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice*. Pearson.
- Schneier, B. (2015). *Applied Cryptography*. Wiley.
- Menezes, A., van Oorschot, P., & Vanstone, S. (1996). *Handbook of Applied Cryptography*. CRC Press.
- Katz, J., & Lindell, Y. (2020). *Introduction to Modern Cryptography*. CRC Press.
- Singh, S. (2000). *The Code Book*. Anchor Books.
- Python Software Foundation. *Python Documentation*